

CONSTITUTION

HelixCode Inner Go Application — Constitution

Version: 1.0.0 **Date:** 2026-05-04 **Scope:** Inner Go application at HelixCode/
within the HelixCode meta-repo **Authority:** Cascaded from root
CONSTITUTION.md (./CONSTITUTION.md) with Go-application-specific addenda

Article I — Identity

This Constitution governs the inner Go application whose source lives at HelixCode/internal/, HelixCode/cmd/, HelixCode/applications/, HelixCode/tests/, etc. — the actual production code that end users invoke. This is the most important governance node in the repository: bluffs, simulations, and placeholder implementations would live here and cause direct user harm.

Article XI — Anti-Bluff and Quality Mandate

§11.9 — Anti-Bluff Forensic Anchor

Verbatim user mandate: “We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

Operative rule: every PASS in this codebase MUST carry positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects regardless of how green the summary line looks.

Tests and Challenges (HelixQA) are bound equally. A Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does.

No false-success results are tolerable. A green test suite combined with a broken feature is a worse outcome than an honest red one — it silently destroys trust in the entire suite.

Cascade requirement: this anchor (verbatim quote + operative rule) MUST appear in every submodule's CONSTITUTION.md / CLAUDE.md / AGENTS.md. Non-compliance is a release blocker. Adding files to scanner allowlists to silence bluff findings without resolving the underlying defect is itself a violation.

Full text: root CONSTITUTION.md Article XI §11.9.

Article XII — Security Mandates

§12.1 (CONST-042) — No-Secret-Leak

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital, transitively or otherwise. All secrets live in .env files (mode 0600) listed in .gitignore. Any leak — to git, logs, build artefacts, screenshots, or external services — is a release blocker until rotated and post-mortemed.

§12.2 (CONST-043) — No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of main/master, or upstream-overwriting operation may be performed without explicit, in-conversation user approval given for that specific operation. Authorization for one push does not extend to subsequent pushes. Bypassing hooks (--no-verify), signature verification (--no-gpg-sign), or protected-branch rules also requires explicit approval.

Repo-specific addenda — Inner Go application

Go module specifics

- Module: dev.helix.code
- Go version: 1.26
- Build: make build → bin/helixcode
- Single test invocation: go test -v -run TestName ./internal/<pkg>
- Integration tests: go test -v -tags=integration -run TestX ./tests/integration/...

- Mocks live ONLY at `internal/mocks/` and may be imported only by `*_test.go` under `internal/<pkg>/`. Per Constitution Rule 5, integration tests must NOT import `internal/mocks`.

Anti-bluff smoke (run before declaring any feature done)

```
grep -rn "simulated\|for now\|TODO implement\|placeholder"
      internal/ cmd/ applications/ && echo "BLUFF FOUND" ||
      echo "clean"
```

Per-feature DOD (Definition of Done) for Phase 1+2 ports

1. Production code at the documented path under `internal/<pkg>/`.
2. Unit test (mocks allowed).
3. Integration test (`-tags=integration`, no mocks).
4. Challenge under `tests/e2e/challenges/<feature>/` with `expected.json`.
5. Challenge runs against `make test-infra-up` and produces runtime evidence.
6. Evidence pasted into commit message body.
7. `scripts/bluff-detector.sh` exits clean on the diff.

Universal mandatory rules (reproduced from root)

1. **No CI/CD Pipelines** — no `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, etc.
2. **No Mocks in Production** — mocks only in unit tests (`*_test.go`).
3. **No HTTPS for Git** — SSH URLs only (`git@github.com:...`).
4. **No Manual Container Commands** — use `make build/orchestrator` binary.
5. **Real Data for Non-Unit Tests** — real databases, real HTTP calls, real containers.
6. **100% Challenge Coverage** — every component needs Challenge scripts.
7. **Reproduction-Before-Fix** — every bug reproduced by Challenge before any fix.
8. **Definition of Done** — done requires pasted terminal output from real run against real artifacts.
9. **No Self-Certification** — no *verified/tested/working/complete/fixed/passing* without pasted command output.
10. **Zero-Bluff Mandate (CONST-035)** — every test must guarantee Quality + Completion + Full Usability.

See also

- Root Constitution: `../CONSTITUTION.md`

- Root CLAUDE.md: ../CLAUDE.md
 - Root AGENTS.md: ../AGENTS.md
 - Synthesis spec: ../docs/superpowers/specs/2026-05-04-cli-agent-fusion-synthesis-design.md
 - Phase 0 plan: ../docs/superpowers/plans/2026-05-04-phase-0-foundation-cleanup.md
 - Progress tracker: ../docs/improvements/PROGRESS.md
-

CONST-045 — No Hardcoded Distribution Hosts (cascaded from root CONSTITUTION.md)

ALL container distribution targets SHALL be configured exclusively through the `CONTAINERS_REMOTE_HOST_N_*` environment variables in `containers/.env`. NO host (hostname, IP, user, key path, runtime, label) may be hardcoded in ANY source file, test, challenge, config template, script, or governance document. Adding/removing hosts = editing `containers/.env` only; NO code change is permitted. Tests SHALL read `.env` at runtime and skip with `SKIP-OK`: marker when `CONTAINERS_REMOTE_ENABLED=false`. See root `CONSTITUTION.md` §CONST-045 for the full mandate, audit command, and cascade requirements.

CONST-047 — Recursive Submodule Application Mandate (cascaded from root CONSTITUTION.md)

Verbatim user mandate (2026-05-14): “Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!”

Every engineering deliverable produced for the main project MUST be applied — fully and recursively — to every owned submodule under the `vasic-digital` and `HelixDevelopment` GitHub organizations. Each owned submodule (including this one) MUST receive in lockstep: (1) anti-bluff posture (CONST-035 / Article XI §11.9), (2) comprehensive documentation matching actual capabilities, (3) full tests + Challenges coverage with captured runtime evidence, (4) recursive propagation through nested submodules under the same orgs, (5) synchronized commits when meta-repo state advances this surface.

See the root `CONSTITUTION.md` §CONST-047 for the full mandate. This anchor MUST remain in this submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`.

CONST-048 — Full-Automation-Coverage Mandate (cascaded from root CONSTITUTION.md)

Verbatim user mandate (2026-05-15): *“Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered!”*

No feature/functionality/flow/use-case/edge-case/service/application on any supported platform of this Go application is deliverable until covered by automation tests proving six invariants: anti-bluff posture with captured runtime evidence (CONST-035), proof of working capability end-to-end on target topology, implementation matching documented promise, no open issues/bugs surfaced, full documentation in sync, four-layer test floor.

See root CONSTITUTION.md §CONST-048 and constitution submodule Constitution.md §11.4.25 for the full mandate.

CONST-049 — Constitution-Submodule Update Workflow Mandate (cascaded from root CONSTITUTION.md)

Verbatim user mandate (2026-05-15): *“Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE committed and pushed to all constitution Submodule upstreams!”*

7-step pipeline before any constitution-submodule edit. See root CONSTITUTION.md §CONST-049 and constitution submodule Constitution.md §11.4.26 for the full mandate.

CONST-050 — No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from root CONSTITUTION.md)

Verbatim user mandate (2026-05-15): *“Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test*

types MUST interact with real fully implemented System! ... All codebase of the project MUST BE 100% covered with every supported test type.”

(A) Mocks/stubs/fakes/placeholders/TODOs/FIXMEs/“for now” patterns PERMITTED only in unit-test sources; non-unit tests MUST exercise this Go application against real infrastructure. Production code MUST NOT import `internal/mocks/`. **(B)** 100% test-type coverage: unit + integration + E2E + full-automation + security + DDoS + scaling + chaos + stress + performance + benchmarking + UI + UX + Challenges (`../challenges/`) + `helix_qa` (`../helix_qa/`).

See root `CONSTITUTION.md` §CONST-050 and constitution submodule `Constitution.md` §11.4.27 for the full mandate.

CONST-051 — Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from root `CONSTITUTION.md`)

Verbatim user mandate (2026-05-15): “All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory) are equal parts of the project’s codebase! ... We MUST NEVER modify Submodules to bring into them any project specific context ... All Submodule dependencies that are used by Submodule MUST BE accessed from the root of the project! We MUST NOT have nested Submodule dependencies.”

(A) Every owned-by-us submodule is an EQUAL part of this Go application’s codebase. Same engineering attention as main: analysis, extension, tests, gap-fill, bug-fix, documentation. **(B)** Submodules MUST stay fully decoupled — NEVER inject project-specific context. **(C)** Dependencies of owned submodules MUST live at parent project’s root (`../<name>/` or `../submodules/<name>/`); nested own-org submodule chains FORBIDDEN. Third-party submodules exempt.

See root `CONSTITUTION.md` §CONST-051 and constitution submodule `Constitution.md` §11.4.28 for the full mandate. ### CONST-052 — Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29) Every directory/submodule/file MUST use lowercase snake_case names. Existing non-compliant names MUST be renamed atomically with updates to all references (configs, docs, source-code imports, governance files). Common-sense exceptions: language-mandated case (Java/Kotlin/Android/Apple/C#/Swift) inside language-root, vendor third-party submodules, build artefacts. `Upstreams/` → `upstreams/` transition: `install_upstreams.sh` supports BOTH directory names during migration. Phased execution; each rename batch ships with (i) reference-

resolution regression test, (ii) full CONST-050(B) test-type matrix run, (iii) anti-bluff wire-evidence. See root CONSTITUTION.md §CONST-052 and constitution submodule Constitution.md §11.4.29 for the full mandate.

CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

Verbatim user mandate (2026-05-15): “every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be committed every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!”

Every project module, owned-by-us submodule, service, and application MUST ship a proper .gitignore. Forbidden-from-version-control classes:

1. **Build artefacts:** /bin/, /build/, /dist/, /out/, target/, *.exe, *.dll, *.so, *.dylib, *.a, *.o, *.class, *.pyc, generator-produced files when the generator is committed.
2. **Cache files:**
__pycache__/, .pytest_cache/, .mypy_cache/, .ruff_cache/, node_modules/, .next/, .cache/, .gradle/, .terraform/, language-server caches.
3. **Temp files:** *.tmp, *.swp, *~, .DS_Store, Thumbs.db, *.orig, *.rej.
4. **Sensitive-data files:** .env, .env.* (allow .env.example placeholder only — no real secrets even as examples), *.pem, *.key, *.crt, id_rsa*, id_ed25519*, .netrc, secrets/, api_keys.sh.
5. **Generated reports/logs:** *.log, coverage.out, htmlcov/, runtime captures unless reference assets.
6. **OS/IDE personal state:** .idea/, .history/, .vscode/ (except shared settings).

Anti-bluff invariant: .gitignore line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked *.log despite the ignore-line is a violation of equal severity to no ignore-line at all.

Pre-commit attention: every commit author (human OR agent) MUST inspect `git diff --staged + git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (un-stage, add to .gitignore, scrub if already-tracked). Gate CM-GITIGNORE-PRECOMMIT-AUDIT + paired mutation.

Secret-leak intersection (CONST-042 / §11.4.10): a `.env` leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

Recreatable-content test: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

Cascade requirement: This anchor (verbatim or by CONST-053 ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule `Constitution.md` §11.4.30 for the full mandate.

CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

Verbatim user mandate (2026-05-15): *“We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls perhaps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to everyone.”*

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}], transitive_handling: {recursive, conflict_resolution}, language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/ .helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

Cascade requirement: This anchor (verbatim or by CONST-054 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule Constitution.md §11.4.31 for the full mandate.

CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

Verbatim user mandate (2026-05-15): “Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!”

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke, etc.) against the post-pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: Reopened, Type: Bug); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

Cascade requirement: This anchor (verbatim or by CONST-055 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to §11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule Constitution.md §11.4.32 for the full mandate.

CONST-056: Mandatory install_upstreams on clone/ add Mandate (cascaded from constitution submodule §11.4.36)

Verbatim user mandate (2026-05-15): “Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository’s upstreams!”

Every clone / add of a Git repository under HelixCode MUST be followed by install_upstreams invocation from the repository’s root IF its tree contains upstreams/ (or legacy Upstreams/ per CONST-052 transition) populated with *.sh recipe files. The utility (installed on operator’s PATH via .bashrc/.zshrc; implementation in the constitution submodule’s install_upstreams.sh — already supports BOTH directory names since constitution commit 45d3678) reads the recipe files, configures every declared upstream as a named git remote, and fans out origin push URLs.

Skipping the invocation when upstreams/ is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate CM-INSTALL-UPSTREAMS-ON-CLONE + paired mutation. Automation: the future incorporate-submodule per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: git remote -v | grep -c push reports expected count.

Cascade requirement: This anchor (verbatim or by CONST-056 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.36 for the full mandate.

CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal **Status:** value, drawn from this 3-element closed map:

Item **Type:**	Closure **Status:** value
Bug	Fixed (→ Fixed.md)

Item **Type:**	Closure **Status:** value
Feature	Implemented (→ Fixed.md)
Task	Completed (→ Fixed.md)

The (→ Fixed.md) suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (in HelixCode: the §11.4.93 workable-items export DB exporter — which SUPERSEDES the markdown-derived generate_issues_summary.sh / generate_fixed_summary.sh, both of which now REFUSE to run — and the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all “closed, positive evidence captured”) while preserving the literal in the emitted document.

Closing a Feature with Fixed (→ Fixed.md) or a Task with Implemented (→ Fixed.md) is a CONST-057 violation. Gate CM-CLOSURE-VOCAB-TYPE-AWARE walks every Fixed.md heading + every Issues.md heading whose ****Status:**** is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

Cascade requirement: This anchor (verbatim or by CONST-057 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.33 for the full mandate.

CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose ****Status:**** is Reopened MUST carry, within 8 non-blank lines of the heading, a ****Reopened-Details:**** line capturing four sub-facts:

- **By:** AI or User (source-of-truth observer who flipped the status). AI covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). User covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (YYYY-MM-DD).
- **Reason:** one-line cause classification — chosen from the closed vocabulary { test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered }. Other values permitted with explicit Reason: <free text> annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens

without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The `Issues_Summary.md` Status column MUST distinguish the four Reopened sub-states by source so a sweep query for “reopens by AI in the last 30 days” is mechanically possible. Suggested column rendering: Reopened (AI: test-failed) vs Reopened (User: manual-testing). Gate CM-ITEM-REOPENED-DETAILS mirrors CM-ITEM-OPERATOR-BLOCKED-DETAILS (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

Cascade requirement: This anchor (verbatim or by CONST-058 ID reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.34 for the full mandate.

CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule’s** three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project’s **repository-root files** (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md`) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md import` or the portable `## INHERITED FROM constitution/CLAUDE.md` heading). They contain only project-specific rules per §11.4.17.

When in doubt about which file to edit: universal rule → constitution submodule’s file; project-specific rule → consumer’s file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

Terminology: “the parent CLAUDE.md” / “the root Constitution” → constitution-submodule file at `constitution/<filename>`; “the project CLAUDE.md” / “this project’s AGENTS.md” → consumer-side file at `<project-root>/<filename>`.

No silent demotion or silent promotion. Moving a rule between layers MUST be a visible commit — `git mv` of a section if it’s a clean clone, or explicit Lifted from `<project>` to constitution per §11.4.35 / Demoted from constitution to `<project>` per §11.4.35 commit-message annotation.

Gate CM-CANONICAL-ROOT-CLARITY verifies (a) consumer’s `CLAUDE.md` opens with the inheritance pointer, (b) constitution submodule’s three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution

submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

Cascade requirement: This anchor (verbatim or by CONST-059 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.35 for the full mandate.

CONST-062: README always-sync mandate (cascaded from constitution submodule §11.4.59)

Verbatim user mandate (2026-05-19): “fully review and update our main README document. Some points are not valid anymore, some are missing. Make sure main README is among documents we MUST ALWAYS keep updated and in Sync with the projects and other documentation! Make sure we always export it (on every update) into PDF and HTML. This mandatory rules/constraints MUST BE all added into the root (constitution Submodule) Constitution, AGENT.MD and CLAUDE.MD!”

README.md at the project root is a §11.4.12-class always-sync document. It MUST be (1) reviewed and updated whenever new docs / integrations / Status.md entries appear, new submodules land, applied-fixes count changes, or canonical paths shift; (2) kept in lockstep with docs / CONTINUATION.md (§12.10) and the Issues / Issues_Summary / Fixed / Fixed_Summary doc set; (3) exported to .html and .pdf on every update via scripts/testing/sync_readme_export.sh (pandoc + weasyprint); (4) carry a §11.4.44 revision header; (5) contain a Documentation Map section linking to every Status.md + Status_Summary.md + spec + plan + guide + script-companion doc + changelog + the constitution submodule, plus per-audience navigation; (6) self-contained (no hyperlinks to ephemeral external systems as the only source of truth). Pre-build gate CM-README-EXPORT-SYNC locks four invariants (README.md exists, README.html exists, README.html mtime ≥ README.md mtime, README.pdf mtime ≥ README.md mtime). Paired meta-test mutation backdates HTML+PDF → gate FAILs. No escape hatch — no --skip-readme-sync, --no-readme-export, --readme-stale-OK flag.

Cascade requirement: This anchor (verbatim or by CONST-062 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.59 for the full mandate.

CONST-063: Documentation always-sync composite covenant (cascaded from constitution submodule §11.4.60)

Verbatim user mandate (2026-05-19 ~09:00Z): *“Double check if all documents are properly tied with our root Constitution, CLAUDE.MD and AGENTS.MD so they are always up to date, always in sync and exported into PDF and HTML! ... Issues, Issues_Summary, Fixed, Fixed_Summary, Continuation, Status and Status_Summary for all contexts (areas) — THEY ALL MUST BE REGULARLY UPDATED, IN SYNC AND CONSISTENT without giving at any moment false picture about the state of the project or particular area(s) of it!”*

Eight documentation classes constitute the project’s living state surface and MUST be in sync at all times across .md + .html + .pdf artefacts: (1) docs/Issues.md, (2) docs/Issues_Summary.md, (3) docs/Fixed.md, (4) docs/Fixed_Summary.md, (5) docs/CONTINUATION.md, (6) README.md, (7) every docs/**/Status.md (domain-scoped), (8) every docs/**/Status_Summary.md (domain-scoped). Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 each govern individually; §11.4.60 binds them via single composite gate CM-DOCS-COMPOSITE-SYNC (pre-build) that FAILs the build if ANY single instance’s .html or .pdf mtime is older than .md mtime. Walks docs/ recursively for the Status fleet. Paired mutation backdates docs/Issues.html to year 2000 → gate FAILs. No escape hatch — no --skip-composite-doc-sync, --allow-stale-html, --summary-not-applicable flag exists.

Cascade requirement: This anchor (verbatim or by CONST-063 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.60 for the full mandate.

CONST-064: Mandatory Markdown metadata table + structured-doc ToC (cascaded from constitution submodule §11.4.61)

Verbatim user mandate (2026-05-19): *“For every Markdown document which contains structured content (with headings / sections and sub-sections) make sure that every time we apply change to the structure, table of contents on the top of the document is created or updated! This is MANDATORY for every structured MARKDOWN document. Automatically its PDF and HTML versions MUST BE (re)generated!”*

Introduce ... revision number, date and time of creation, date and time of last modification, other useful information we have in documents such as Issues, Issues_Summary, Fixed, Fixed_Summary, Status, Status_Summary, Continuation and similar. ... make this mandatory for EVERY Markdown document from now on, update root constitution Submodule with these changes and commit and push it all to all upstreams.”

Verbatim 2026-05-19 operator mandate: “all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

Every tracked *.md in the §11.4.65 INCLUDED set MUST carry, immediately below the H1 title, a canonical Markdown metadata table with four MANDATORY rows (Revision — monotonic positive integer; Created — ISO 8601 date; Last modified — ISO 8601 date; Status — active/draft/deprecated/superseded) plus ENCOURAGED rows (Status summary, Issues, Issues summary, Fixed, Fixed summary, Continuation) rendered with –/none when N/A. Pre-existing §11.4.44 bold-line headers remain historically valid but MUST migrate to the table at the next substantive edit (30-day migration window from §11.4.61 landing). Any tracked *.md with **two or more H2 sections** MUST include a ## Table of contents section immediately after the metadata table; the ToC MUST list every H2/H3 in document order with anchor links and MUST be regenerated on every structural change (heading added/removed/renamed/reordered). A stale ToC is a §11.4 PASS-bluff: operators reading the rendered Markdown see a navigation map that no longer matches the body. Narrow exemptions: LICENSE, LICENSE.md, NOTICE, VERSION, OWNERS, machine-generated CHANGELOG.md, plus the §11.4.65 EXCLUDED set. Anti-bluff gates CM-MD-METADATA-PRESENT (walks every tracked *.md minus exemptions; asserts H1 + 4 MANDATORY rows present within 25 lines below H1) + CM-MD-TOC-PARITY (for every *.md with ≥2 H2 sections asserts ## Table of contents exists and its slugs match live H2/H3 set in order). Paired mutations strip metadata table → CM-MD-METADATA-PRESENT FAILs; rename one H2 without regenerating ToC → CM-MD-TOC-PARITY FAILs. The metadata or ToC change is a .md modification, which transitively triggers §11.4.65 regeneration of .html/.pdf siblings — the two clauses compose without duplicate enforcement.

Cascade requirement: This anchor (verbatim or by CONST-064 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.61 for the full mandate.

CONST-066: Universal Markdown export mandate (cascaded from constitution submodule §11.4.65)

Verbatim user mandate (2026-05-19): “Any markdown document inside the project and which is not part of the applications or services source code MUST BE exported (be available) in PDF and HTML! Any already existing Markdown document that fulfills this condition and which does not have HTML or PDF at all or it is not in sync with it MUST HAVE (re)generated PDF and HTML version! Every time when Markdown document (file) is modified, its proper HTML and PDF versions MUST BE regenerated. Markdown documents MUST BE at all times in sync with PDF and HTML versions!”

Verbatim 2026-05-19 operator mandate: “all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”

Every Markdown document inside the project that is NOT part of an application or service’s source-code tree MUST have synchronized .html and .pdf siblings, all three artefacts in sync at all times. **INCLUDED scope (closed-set):** project-root *.md (README.md, CLAUDE.md, AGENTS.md, CONTRIBUTING.md, etc.); docs/**/* .md (guides, research, plans, changelogs, procedures, hardware notes); scripts/**/* .md (script-companion docs in documentation format, NOT shebang scripts); owned-submodule trees top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and any docs/**/* .md; constitution/**/* .md (the canonical-root submodule); owned HelixQA-set dependencies’ equivalents. **EXCLUDED:** external/**, prebuilts/**, packages/modules/**, kernel-*/**, out/**, build/**, application/service source-code trees, third-party submodules NOT in the owned set. Mandatory protections (ALL must hold): (1) every INCLUDED .md has .html + .pdf siblings — a missing export is a §11.4.65 violation regardless of when the markdown was last touched; (2) .html/.pdf mtime ≥ .md mtime within the same sync-wrapper invocation granularity — stale exports are violations even if the .md itself is correct; (3) every modification triggers regeneration via scripts/testing/sync_all_markdown_exports.sh (or per-class wrappers that delegate to the same canonical helper); (4) pre-build gate CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC walks the INCLUDED scope and FAILs the build if any are missing or stale. Canonical helper invokes pandoc (HTML) + weasyprint (PDF) with timeout 60 each, uses docs/_progress-style.css for visual consistency, supports --check-only and --regenerate-all modes, caps at 500 candidates with explicit abort+list if scope is unexpectedly

large. No escape hatch — no --skip-md-exports, --no-pdf-only, --md-export-not-applicable, --application-internal-doc flag exists for files inside the INCLUDED scope.

Cascade requirement: This anchor (verbatim or by CONST-066 ID reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.65 for the full mandate.

CONST-067: Blocker-resolution interactive-clarification mandate (cascaded from constitution submodule §11.4.66)

Verbatim user mandate (2026-05-19): *“If any blockers which can be resolved with interactive response ever happen again, perform in depth research on options doable by your side and how much inputs from us you really need, then create options and present them to us. After we answer, preferably you will be unblocked and be able to continue work on blocked items. Let us make this main approach when such situations (blockers) do happen!”*

Verbatim 2026-05-19 operator mandate: *“all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”*

When any task is blocked (operator decision, hardware access, external-service authorization, ambiguous scope, or any input the agent cannot mechanically derive), the agent MUST follow this five-step discipline before idling or asking free-form questions: (1) **In-depth research on agent-side options** — identify the maximum scope that can land unilaterally without operator input; research the codebase, upstream documentation, existing tooling, current state, involved files/components, existing tests/gates; surface every agent-actionable path even if it requires a small operator confirmation. (2) **Calculate minimum-viable operator input** — reduce the question set to the smallest closed set of operator-only decisions (preference, authorization, scope-bounding, hardware availability, schedule); anything that can be researched MUST NOT be asked. (3) **Construct 2–4 mutually-exclusive options** — each with (a) short label, (b) one-line trade-off description, (c) explicit statement of what the agent will do *after that answer*; one marked “Recommended” with rationale; options MUST genuinely differ — restating one option as three close variants is itself a §11.4.66 violation. (4)

Present via the platform’s interactive question mechanism — on Claude Code that is AskUserQuestion (max 4 questions, each with 2–4 options; supports multi-select for non-mutually-exclusive sets); other platforms (Copilot CLI, Codex, Gemini CLI) use their equivalents; NEVER inline free-text “what would you like?” when interactive options would do — that wastes operator attention. (5) **After the operator answers, resume work without additional round-trips** — every option’s promised action MUST be sufficient to unblock; if a follow-up clarifying question is needed, the prior options were insufficiently researched and that’s itself a §11.4.66 violation. The contract is: ask once, unblock, continue. Composes with §11.4.6 (no-guessing — interactive options replace agent guessing about operator preference) / §11.4.7 (demotion-evidence — operator preference is captured-evidence for direction changes) / §11.4.40 (full-suite retest — uninterrupted work post-answer means the test cycle resumes cleanly without context drift) / §11.4.41 (merge-first — when operators make conflicting choices across sessions, the merge-first discipline preserves both) / §11.4.42 / §11.4.52. Pre-build gate CM-COVENANT-114-66-PROPAGATION enforces the anchor literal across the consumer fleet. Paired meta-test mutation strips the literal → gate FAILs. No escape hatch — no --skip-ask, --silent-wait, --free-form-only flag.

Cascade requirement: This anchor (verbatim or by CONST-067 ID reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. See constitution submodule Constitution.md §11.4.66 for the full mandate.

CONST-068: Shell-script target-shell-parseability mandate (cascaded from constitution submodule §11.4.67)

Verbatim user mandate (2026-05-19): *“any issue we spot must be fixed, bash scripts as well if they are broken!” + “Make sure that this is mandatory rule!”*

Verbatim 2026-05-19 operator mandate: *“all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!”*

Every committed shell script MUST be parseable by its target interpreter (sh -n for /bin/sh, bash -n for /bin/bash, etc.) AND MUST declare a shebang matching its actual syntax usage. Bash-only constructs (>(. . .), <(. . .), [[]], <<<, arrays, \${var^^}, etc.) used in scripts that may be invoked via sh

`script.sh` MUST be wrapped in `eval` so the parser sees only a string (target shells like `mksh` parse the entire script before executing — runtime guards cannot save a parse-time rejection). Honest shebangs only: `#!/bin/bash` only if `bash` actually expected; `#!/bin/sh` requires POSIX-clean body. Fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51. Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

Cascade requirement: This anchor (verbatim or by `CONST-068` ID reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.67 for the full mandate.

§11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

Verbatim user mandate (2026-05-20): *“We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!”*

A test that asserts audio or video routing **PASS** MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing **PASS**: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from `/proc/asound/.../status hw_ptr`; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) `ffprobe-on-captured-mp4` with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) `tinycap` RMS amplitude above the line-level floor. Empty / `<unreachable>` / `<N.E.>` / `<None>` placeholders are NOT positive evidence; a missing-but-required sink is `OPERATOR-BLOCKED` (release-blocker), never `SKIP`, never `PASS`. No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag exists.

Cascade requirement: This anchor (verbatim or by §11.4.68 reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 **PASS**-bluff at the sink-side-evidence

layer. **Canonical authority:** constitution submodule Constitution.md §11.4.68 for the full mandate.

§11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

Verbatim user mandate (2026-05-20): *“Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!”*

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per superpowers : subagent-driven-development. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution` are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

Cascade requirement: This anchor (verbatim or by §11.4.70 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule Constitution.md §11.4.70 for the full mandate.

§11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

Verbatim user mandate (2026-05-20): *“before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brought in MUST BE incorporated,*

covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!”

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (`git fetch --all --prune --tags`, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto--ours/--theirs); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit’s body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with §11.4.4(b) four-layer coverage + §11.4.43 TDD-fix discipline, every PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with `git ls-remote post-push`. No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag.

Cascade requirement: This anchor (verbatim or by §11.4.71 reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the push-discipline layer.
Canonical authority: constitution submodule `Constitution.md` §11.4.71 for the full mandate.

§11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

Verbatim user mandate (2026-05-20): *“Make sure all fixes for audio are always top priority in main working stream!”*

The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no “but this non-audio task is faster” or “but this research is more interesting” override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

Cascade requirement: This anchor (verbatim or by §11.4.72 reference) MUST appear in every owned submodule’s `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation at the dispatch-priority

layer. **Canonical authority:** constitution submodule Constitution.md §11.4.72 for the full mandate.

§11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

Verbatim user mandate (2026-05-20): “Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document. Document MUST BE updated ALWAYS to follow the versioning rules we are applying here + revision and other properties we have!”

Applies **only when a project recognises a main specification document**. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table Revision integer, bumped for every other change); (3) the metadata table MUST stay current (Revision, Last modified, Status summary, Fixed); (4) propagated copies of the rule MUST reference the active specification. V<primary>.md, not a stale archive; (5) on primary bump the old file moves to <spec-dir>/archive/ with Status: superseded. Classification: universal, applicable conditionally per the scope condition.

Cascade requirement: This anchor (verbatim or by §11.4.73 reference) MUST appear in every owned submodule’s CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a release blocker when a project has a main spec and lets it drift. **Canonical authority:** constitution submodule Constitution.md §11.4.73 for the full mandate.

§11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don't-Reimplement (cascaded from constitution submodule §11.4.74)

Verbatim user mandate (2026-05-20): “We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules furter! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!”

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — vasic-digital and HelixDevelopment on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature's tracker entry with a Catalogue-Check: field (reuse <org/repo>@<sha> / extend <org/repo>@<sha> / no-match <date>). Classification: universal.

Cascade requirement: This anchor (verbatim or by §11.4.74 reference) MUST appear in every owned submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. **Canonical authority:** constitution submodule Constitution.md §11.4.74 for the full mandate.